

Discusion: Analisis probabilistico y algoritmos

Integrantes

- Christian Echeverria 221441
- Gustavo Cruz 22779
- Josue Say 22801
- Mathew Cordero 22982
- Pedro Guzman 22111

1 . ¿Que podemos decir acerca del tiempo de ejecucion en diferentes ejecuciones de un algoritmo aleatorizado? ¿Que es lo que podemos medir al analizar un algoritmo aleatorizado?

Un algoritmo aleatorizado tiene un tiempo de ejecucion de estimacion. Esto se debe a la naturaleza volatil de los algoritmos que se analizan. Por lo tanto es como si se calculase una probabilidad sobre el tiempo estimado dado por $E[x]$ donde x sera n en $O(n)$

Hiring Algorithm

```
1      best=nadie
2      for i=1 to n
3          entrevistar candidato i
4          if candidato i es mejor que best
5              best = candidato i
6              contratar(candidato i)
```

Figure 1: alt text

2. ¿Cuales son el best-case y worst-case escenarios para este algoritmo?

Para hacer este analisis analizamos cada linea

Worst Case

En este caso para analizarlo vamos a ir linea por linea

Hiring Algorithm

```
1      best = nadie    --->  $O(1)$ 
```

```

2   for i = 1 to n    -> sumatoria hasta n
3       entrevistar candidato i ----> O(1)
4       if candidato i es mejor que best ----> O(1)
5           best = candidato i ----> O(1)
6           contratar(candidato i) ----> O(1)

```

Teniendo un arreglo de usuarios de tamaño n entonces vamos a definir que

$$T(n) = \sum_{i=0}^n 4O(1) + C$$

$$T(n) = \sum_{i=0}^n O(1)$$

$$T(n) = (n + 1) \cdot O(1)$$

$$T(n) = O(n)$$

Best Case

Para este sabemos que el mejor siempre va a ser el primero ya que la lista viene ordenada de mayor a menor por ende el best case es:

$$O(1)$$

3. ¿De que depende el costo de este algoritmo de solucion para el hiring problem? ¿Cual parte de ese costo podemos calcular directamente y cual no? ¿Que nos impide calcular la parte que no podemos?

- El costo depende exclusivamente de quien es el mejor , y el mejor sera dado por el costo de entrevistar y el costo de contratar a nuestro usuario.

Entonces si lo definimos en variables serian Costo de contratacion (Ch), Costo de entrevista (Ci) , Numero de candidatos (n)

- El costo de entrevistas se calcula directamente, Sabemos el numero de candidatos (n) y podemos estimar el costo de entrevistar a cada uno.
- Lo que no se puede calcular de manera directa es el costo de contratar. Esto porque dependera de cuando encontremos un nuevo mejor candidato, es como cuando se recalcula el valor de los mejores que hemos encontrado por cada nueva corrida. Esto solamente es un valor estimado.

4. Probabilidad de contratar al candidato i y valor esperado de contrataciones

Enunciado

¿Cual es la probabilidad de que el i -esimo candidato o candidata sea contratado?

¿Cual resulta ser, entonces, el valor esperado de la cantidad de contrataciones?

Si suponemos que hay n candidatos y que el orden de llegada es una permutacion aleatoria uniforme. Y luego definimos el evento:

$$C_i = \{\text{el candidato } i \text{ es contratado}\}.$$

- **Probabilidad de contratar al candidato i :** El candidato i sera contratado si y solo si su calidad es la mayor entre los primeros i entrevistados. Dado el supuesto de aleatoriedad, todos los i candidatos tienen la misma probabilidad de ser el mejor de ese subconjunto, asi que

$$\Pr[C_i] = \frac{1}{i}.$$

- **Numero esperado de contrataciones:** Sea X el numero total de contrataciones. Podemos escribir

$$X = \sum_{i=1}^n [C_i],$$

donde $[C_i]$ es la variable indicadora de que el candidato i fue contratado. Por linealidad de la esperanza,

$$\mathbb{E}[X] = \sum_{i=1}^n \mathbb{E}[[C_i]] = \sum_{i=1}^n \Pr[C_i] = \sum_{i=1}^n \frac{1}{i} = H_n$$

siendo H_n el n -esimo numero armonico. En particular,

$$H_n \approx \ln n + \gamma,$$

donde γ es la constante de Euler–Mascheroni.

5. Manifestacion de la aleatorizacion y su efecto en el tiempo de ejecucion

Enunciado

Supongamos que la agencia de reclutamiento toma especiales precauciones contra posibles fraudes. Para evitar que los candidatos se pongan de acuerdo y

saboteen el procedimiento de entrevista para favorecer a alguien, ¿como se manifestaria la aleatorizacion planteada arriba en el algoritmo del hiring problem? ¿Que efecto tendria esto sobre el conteo de operaciones? ¿Que es importante tomar en cuenta acerca de la aleatorizacion sobre el tiempo de ejecucion? ¿Como se manifestaria la aleatorizacion planteada en el algoritmo del hiring problem? ¿Que efecto tendria esto sobre el conteo de operaciones? ¿Que es importante tomar en cuenta acerca de la aleatorizacion sobre el tiempo de ejecucion?

- **¿Como se introduce la aleatorizacion?** Antes de empezar las entrevistas, barajamos la lista de candidatos con un algoritmo de *shuffle* uniforme. Esto fuerza que el orden de llegada sea una permutacion aleatoria, impidiendo manipulaciones o acuerdos previos.
- **Impacto en el conteo de operaciones:**
 1. **Barajado inicial:** El shuffle de Fisher-Yates hace $(n - 1)$ intercambios, es decir, $\Theta(n)$ operaciones.
 2. **Algoritmo de contratacion:** Recorre los n candidatos comprobando si cada uno es mejor que el mejor visto hasta ahora; esto son $\Theta(n)$ comparaciones.

Por tanto, el costo total es

$$T(n) = \underbrace{\Theta(n)}_{\text{shuffle}} + \underbrace{\Theta(n)}_{\text{entrevistas}} = \Theta(n).$$

- **¿Que considerar sobre el tiempo de ejecucion al aleatorizar?**
 - El tiempo deja de ser determinista y pasa a ser una variable aleatoria, pero mantenemos garantizado que siempre sea $\Theta(n)$ en el peor caso y en el caso promedio.
 - Analizamos el tiempo esperado, $\mathbb{E}[T(n)] = O(n)$.
 - Al ser un algoritmo Las Vegas, debemos asegurarnos de que la variabilidad del tiempo sea aceptable para nuestro entorno operativo.

Problema 6

Enunciado:

Provea una cota inferior para el tiempo de ejecucion de Permuted-By-Sorting. ¿Que instrucciones podrian tener impacto significativo sobre el tiempo de ejecucion?

Primero, analicemos el algoritmo Permuted-By-Sorting:

- - $n = A.length$
 - Obtiene la longitud del arreglo A
 - let P[1..n] be a new array - Crea un nuevo arreglo P de longitud n

- for $i = 1$ to n - Itera sobre cada elemento

-

$$P[i] = \text{RANDOM}(1, n^3)$$

- Asigna a cada posicion un numero aleatorio entre 1 y

$$n^3$$

-

$$\text{sort } A, \text{ using } P \text{ as sort keys}$$

- Ordena A usando P como claves de ordenamiento

Para establecer una cota inferior, debemos identificar las operaciones que inevitablemente deben realizarse sin importar la implementacion:

- Generacion de claves aleatorias: El bucle que asigna valores aleatorios a P requiere $\Omega(n)$ operaciones, ya que debe recorrer todo el arreglo P.
- Ordenamiento: La operacion de ordenamiento tiene una cota inferior conocida de $\Omega(n \log n)$ comparaciones para cualquier algoritmo de ordenamiento basado en comparaciones. Aunque solo estamos ordenando las referencias de A segun P, necesitamos hacer estas comparaciones.

Por lo tanto, la cota inferior para el tiempo de ejecucion de Permute-By-Sorting es $\Omega(n \log n)$. Instrucciones con impacto significativo Las instrucciones que tienen un impacto significativo en el tiempo de ejecucion son:

- Generacion de numeros aleatorios: La funcion

$$\text{RANDOM}(1, n^3)$$

puede tener diferentes implementaciones y costos dependiendo del generador de numeros aleatorios utilizado.

- Algoritmo de ordenamiento seleccionado: El algoritmo de ordenamiento usado para ordenar A segun P tiene un gran impacto. Diferentes algoritmos de ordenamiento tienen diferentes constantes ocultas y comportamientos en casos particulares.
- Rango de los numeros aleatorios: El algoritmo usa

$$n^3$$

como limite superior para generar numeros aleatorios. Este rango grande es necesario para minimizar colisiones, pero trabajar con numeros grandes puede aumentar el costo computacional de las comparaciones.

Como indica la pista, el ciclo principal y el ordenamiento dictan el tiempo de ejecucion, siendo el ordenamiento el factor dominante con su cota inferior de

$$\Omega(n \log n)$$

.

Problema 7

Sabemos que un corte es una particion de los vertices en dos conjuntos no vacios (A y B) y que el tamaño del corte es el numero de aristas que conectan vertices en A con vertices en B.

Pensemos en un vertice v con grado minimo

$$\delta(G)$$

(es decir, tiene exactamente

$$\delta(G)$$

aristas conectadas a el):

Si colocamos este vertice v solo en un conjunto A, y todos los demas vertices en B Todas las aristas conectadas a v cruzaran el corte Por lo tanto, este corte tendra un tamaño de

$$\delta(G)$$

Si intentamos hacer un corte mas pequeño, debemos colocar al menos a uno de los vecinos de v en el mismo conjunto que v . Pero esto solo puede aumentar o mantener igual el tamaño del corte, considerando la estructura completa del grafo. Por lo tanto, el tamaño del corte minimo k no puede ser menor que el grado minimo del grafo

$$\delta(G)$$

, es decir:

$$k \geq \delta(G)$$

.

¿Cual seria la cantidad minima de aristas que debe tener el grafo?

Si un grafo tiene n vertices, y cada vertice tiene al menos un grado minimo

$$\delta(G)$$

, entonces:

La suma total de grados seria al menos

$$n \cdot \delta(G)$$

Como cada arista contribuye 2 al total de grados (uno por cada extremo) El numero minimo de aristas seria

$$n \cdot \delta(G)/2$$

Por lo tanto, la cantidad minima de aristas que debe tener el grafo es

$$n \cdot \delta(G)/2$$

.

¿Como se llega a la probabilidad $2/n$ de contraer una arista del corte?

En el algoritmo de Karger:

Comenzamos con n vertices En cada paso, seleccionamos una arista aleatoriamente y la contraemos Continuamos hasta que quedan solo 2 vertices

La probabilidad de que una arista especifica del corte minimo sea contraida en la primera iteracion es:

Total de aristas en el corte minimo: k Total de aristas en el grafo: al menos

$$n \cdot \delta(G)/2$$

, pero generalmente representado como m Probabilidad de seleccionar una arista del corte:

$$k/m$$

Para el caso especifico donde el grafo es minimamente conectado (con grado minimo):

Cada vertice tiene al menos k aristas (ya que

$$k \geq \delta(G)$$

) El total de aristas es al menos

$$n \cdot k/2$$

La probabilidad de elegir una arista del corte seria como maximo

$$k/(n \cdot k/2) = 2/n$$

Por lo tanto, la probabilidad de contraer una arista del corte minimo en una iteracion es a lo sumo

$$2/n$$

, lo que explica por que el algoritmo de Karger tiene una baja probabilidad de exito en una sola ejecucion cuando n es grande, y por que necesitamos ejecutarlo multiples veces

Problema 8

El resultado mencionado muestra que la probabilidad global de obtener un corte minimo en una sola ejecucion del algoritmo de Karger es

$$\Omega(n^{-2}) \text{ o aproximadamente } 2/(n(n-1))$$

Significado para grafos grandes

$$(n \gg 1)$$

:

Probabilidad muy baja de éxito: Cuando n es muy grande, la probabilidad de encontrar un corte mínimo en una sola ejecución se vuelve extremadamente pequeña. Por ejemplo:

Con $n = 100$ vértices, la probabilidad es aproximadamente 0.0002 (0.02%) Con $n = 1000$ vértices, la probabilidad cae a 0.000002 (0.0002%)

Ineficiencia con una sola ejecución: Este resultado significa que confiar en una sola ejecución del algoritmo es prácticamente inútil para grafos grandes, ya que casi con certeza no encontraríamos el corte mínimo. Degradación cuadrática: La probabilidad disminuye de forma cuadrática

$$(n^{-2})$$

con respecto al número de vértices, lo que implica una degradación rápida del rendimiento a medida que el grafo crece.

Precauciones necesarias: La principal precaución que este resultado sugiere es ejecutar el algoritmo múltiples veces:

Ejecuciones independientes: Debemos ejecutar el algoritmo de Karger múltiples veces de forma independiente y quedarnos con el mejor resultado (el corte más pequeño) encontrado. Número de repeticiones: Para garantizar una alta probabilidad de éxito (por ejemplo,

$$\geq 1 - \delta$$

para algún

$$\delta$$

pequeño), necesitamos ejecutar el algoritmo

$$O(n^2 \log n)$$

veces.

Esto se deriva de la fórmula:

$$(1 - 1/n^2)^r \leq \delta$$

, donde r es el número de repeticiones

Uso de variantes mejoradas: Se pueden utilizar variantes del algoritmo como Karger-Stein (también conocido como “Contracción Recursiva”), que mejora la probabilidad de éxito a

$$\Omega(1/\log n)$$

, requiriendo significativamente menos repeticiones. Almacenamiento de estados intermedios: Una precaucion adicional es guardar estados intermedios del proceso de contraccion para poder explorar diferentes caminos de contraccion, especialmente en las etapas finales del algoritmo donde las decisiones tienen mayor impacto.

9. Reforzamiento de la aleatoriedad en QuickSort

1. Reforzar una distribucion aleatoria uniforme sobre el input

- Se puede reordenar aleatoriamente (**shuffle**) los elementos del arreglo de entrada antes de ejecutar el algoritmo.
- Esto asegura que todas las permutaciones del arreglo tengan la misma probabilidad de aparecer, simulando aleatoriedad en la entrada.

2. Eleccion de pivote

- Si ya reordenamos el input de manera aleatoria, **no es necesario** elegir el pivote de manera aleatoria en cada recursion.
- Podemos simplemente elegir el primer o ultimo elemento como pivote para cada paso, ya que la aleatorizacion de la entrada nos garantiza un comportamiento generalmente razonable.

3. Aleatorizar el QuickSort sin forzar la aleatoriedad del input

- Podemos elegir un pivote de manera aleatoria en cada llamada recursiva, seleccionando un indice aleatorio del subarreglo actual.
- Esto convierte el algoritmo en un **QuickSort aleatorizado**, asegurando buena eficiencia promedio incluso con un input no aleatorio.

10. Variables aleatorias indicadoras en QuickSort

Definicion de las variables indicadoras

Para $1 \leq i < j \leq n$, se define la variable aleatoria indicadora:

$$X_{ij} = \begin{cases} 1 & \text{si los elementos } y_i \text{ y } y_j \text{ son comparados durante QuickSort} \\ 0 & \text{en caso contrario} \end{cases}$$

Formula para X

X es el numero total de comparaciones realizadas por el algoritmo:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

Esta doble suma recorre todas las parejas ordenadas (i, j) tal que $i < j$, contando una unidad si esa pareja se compara.

Explicacion

- En QuickSort, dos elementos se comparan **solamente** si uno de ellos se elige como pivote en la sublista que los contiene a ambos.
- La cantidad esperada de comparaciones es:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$$

Donde $E[X_{ij}]$ es la probabilidad de que y_i y y_j se comparen.

Esta probabilidad es igual a $\frac{2}{j-i+1}$, lo cual se deduce considerando que: > Para que se comparen, ningun elemento entre y_i y y_j debe ser elegido como pivote antes que ellos.

11. ¿Por que la eleccion de un pivote fuera del intervalo $[y_i, y_j]$ no influye en la probabilidad de que estos elementos sean comparados? ¿Cuantos elementos hay en el intervalo $[y_i, y_j]$?

Respuesta:

La eleccion de un pivote **fuera del intervalo $[y_i, y_j]$** (es decir, menor que y_i o mayor que y_j) no afecta la posibilidad de que **y_i y y_j se comparen**, porque ambos seguiran en la misma sublista despues de esa particion. Solo se separaran cuando el pivote elegido este dentro del intervalo (es decir, que divida a y_i y y_j).

Dado que Quicksort es recursivo, solo se realiza la comparacion **si ambos permanecen en la misma sublista hasta que uno de ellos sea elegido como pivote**. Asi, el evento de comparacion depende exclusivamente de que **ningun elemento del intervalo $(y_i, \dots, y_j)$** sea elegido como pivote **antes** que y_i o y_j .

El numero de elementos en el intervalo $[y_i, y_j]$ es:

$$j - i + 1$$

Por lo tanto, la **probabilidad de que y_i y y_j sean comparados** es:

$$E[X_{ij}] = \frac{2}{j - i + 1}$$

Esta probabilidad se deriva del hecho de que solo y_i o y_j deben ser seleccionados como pivote antes que cualquier otro elemento entre ellos.

12. ¿Sera igual la cota del tiempo de ejecucion para la version no aleatorizada? ¿Por que?

Respuesta:

No, no sera igual en todos los casos.

En la version no aleatorizada, si siempre se elige un pivote en una posicion fija (como el primer o ultimo elemento), es posible construir entradas especificas que **siempre generen el peor caso**: sublistas altamente desbalanceadas. Esto lleva a un tiempo de ejecucion de:

$$O(n^2)$$

En cambio, en la version aleatorizada o con analisis probabilistico (asumiendo permutacion aleatoria del input), se garantiza que en promedio el pivote estara cerca del centro del subarreglo, y por tanto, se logra un rendimiento de:

$$E[X] = O(n \log n)$$

Ademas, la aleatorizacion tiene la ventaja de **proteger al algoritmo contra entradas maliciosamente diseñadas** para forzar el peor caso, algo que no es posible en la version determinista.

13. ¿Que impacto tendra sobre el tiempo de ejecucion el revolver el input antes de proceder con el analisis probabilistico? Investigue sobre el tiempo de ejecucion de los generadores de numeros pseudo-aleatorios.

Respuesta:

Revolver el input (es decir, aleatorizarlo antes de aplicar Quicksort) transforma el analisis determinista en uno probabilistico. El impacto clave es que **reduce el riesgo de caer en el peor caso**, lo que estabiliza el rendimiento del algoritmo.

El costo adicional por aleatorizar es generalmente **lineal**, $O(n)$, usando algoritmos como **Randomize-In-Place**, que tiene tiempo esperado eficiente y no requiere ordenamiento. Ademas, revolver el input es equivalente (en distribucion) a seleccionar pivotes aleatorios, por lo que se conserva el analisis probabilistico.

Sobre los **generadores de numeros pseudoaleatorios (PRNG)**:

- Son algoritmos diseñados para producir secuencias de numeros que **simulan ser aleatorios**, pero que son generados de manera determinista a partir de una semilla inicial.
- Los mas comunes, como **rand()** o **random()**, suelen estar basados en **metodos de congruencia lineal**, que permiten generar rapidamente un

nuevo numero pseudoaleatorio usando operaciones simples (suma, multiplicacion y modulo).

- Debido a su simplicidad, la generacion de cada numero pseudoaleatorio requiere solo unas pocas instrucciones de maquina, lo cual significa que operan en **tiempo constante** $O(1)$ o a lo sumo **logaritmico** en algunos casos menos comunes.
- Por tanto, **el costo de generar estos numeros es muy bajo** comparado con el resto del algoritmo Quicksort, cuyo tiempo total de ejecucion esperado es $O(n \log n)$. Incluso si se generan n numeros aleatorios, el costo agregado sigue siendo **lineal**, es decir, $O(n)$.

Conclusion: Revolver el input cuesta poco, pero **mejora significativamente** el comportamiento esperado del algoritmo, evitando el peor caso y garantizando un rendimiento estable.